

MARINE INTELLIGENCE PLATFORM

MarisAI

The Complete Guide — from the idea, to daily use, to the engineering underneath. Written for teammates, judges, and engineers alike.

Live ocean & atmospheric intelligence over Copernicus Marine, Global Fishing Watch, GEBCO, and Open-Meteo · Interactive map, analytics dashboard, data downloader, AI assistant, forecasting engine, and offline ML pipelines

Version: August 2026 build · Prepared for internal team use and hackathon review

Table of Contents

Part I — The Idea

1. What MarisAI Is	01
2. The Problem	02
3. Who It's For	03
4. What Makes It Different	04
5. A Two-Minute Tour	05

Part II — User Guide

6. Getting Around the App	06
7. The Interactive Map	07
8. Reading Map Layers, Plainly	08
9. The Ocean Intelligence Dashboard	09
10. Per-Variable Metric Pages	10
11. The Universal Ocean Data Downloader	11
12. Point Brief & Compare	12
13. The Ocean Assistant (AI Chat)	13
14. Feedback & Contact	14

Part III — Technical Architecture

15. System Overview & Stack	15
16. Backend Conventions	16
17. Data Sources & Providers	17
18. The Map Subsystem	18
19. Vector-Field Particle Engine	19

20. Detectors: Eddies, Upwelling, eDNA	20
21. The Forecasting Engine	21
22. The Forecast Map (Global Grids)	22
23. The Ocean Assistant, Architecturally	23
24. The Universal Downloader, Architecturally	24
25. The Dashboard Backend	25
26. Offline Machine Learning Pipelines	26
27. Frontend Architecture	27

Part IV — Setup, Development & Operations

28. Running MarisAI Locally	28
29. Configuration & Credentials	29
30. Known Limitations & Roadmap	30

Part V — Appendix

31. Data Source Directory	31
32. Glossary	32
33. Frequently Asked Questions	33

PART I

The Idea

What MarisAI is, the problem it answers, and why it exists — written for someone meeting it for the first time, in a demo room or a judging panel.

What MarisAI Is

MarisAI is a marine intelligence platform. It takes the ocean's raw, scattered, expert-only data — satellite sea-surface temperature, wind and current fields, seafloor depth, fishing-vessel activity, species occurrence records, biogeochemistry — and turns it into something a person can actually look at, ask questions of, and act on. Not a research dataset. Not a slide deck. A live, explorable, three-dimensional picture of the ocean, running in a browser.

Concretely, it is three things working together:

An interactive ocean map

A globe you can spin, zoom, and layer with real, live data: temperature, wind, currents, waves, chlorophyll, salinity, biodiversity sampling coverage, detected ocean eddies, coastal upwelling, and seven-day AI forecasts of almost every one of those fields.

An analytics dashboard and forecasting engine

Global KPIs (sea-surface temperature, bleaching risk, heat-stress extent), per-variable deep-dive pages with history and forecast charts, and a machine-learning engine that predicts what a variable will look like up to a month out — everywhere on Earth, not just at a few weather buoys.

A conversational assistant and data toolkit

An AI chat assistant that can call the platform's own tools to answer questions about a location or a phenomenon, a point-and-click "brief" that turns any coordinate into a readable report, a side-by-side comparison between two points, and a universal downloader that exports any of 34 ocean variables as CSV, JSON, or a PDF report for a chosen place and time.

Underneath, two offline machine-learning pipelines — a harmful-algal-bloom early-warning system and a fish-habitat suitability model — run on a shared data-fusion layer that regrids and aligns every source onto common coordinates, so a model can be trained on ocean physics, biology, and bathymetry together rather than one dataset at a time.

FOR JUDGES, IN ONE SENTENCE

MarisAI is a single platform that makes live, global ocean data browsable, explainable, forecastable, and exportable — for a scientist, a fisher, a policy analyst, or a curious teammate — without any of them needing to touch a NetCDF file, a Python notebook, or a paid data subscription.

The Problem

Ocean data is not scarce. It is abundant, public, and almost unusable by anyone outside oceanography. Four frictions repeat everywhere:

1. The data is fragmented across a dozen institutions

Sea-surface temperature and currents live at Copernicus Marine. Fishing vessel activity lives at Global Fishing Watch. Seafloor depth lives at GEBCO. Weather and marine forecasts live at Open-Meteo. Species sightings live at OBIS/GBIF. Coral bleaching risk lives at NOAA Coral Reef Watch. Each has its own API, its own file format (often NetCDF or GRIB — binary scientific formats no browser can read), its own units, its own coordinate convention, and its own outage behaviour. Building anything that uses more than one of these normally means becoming an expert in all of them first.

2. The tooling assumes a scientist is already at the keyboard

The standard way to "use" this data is a Python or MATLAB script that downloads a multi-gigabyte file, opens it with a specialist library, and plots a static image. That is a fine workflow for a researcher who already knows what they're looking for. It is a wall for a teammate, a stakeholder, or a judge who wants to ask "what does the water look like off this coast right now" and get an answer in the next ten seconds, not after installing ``xarray``.

3. Point measurements do not answer "everywhere" questions

Weather buoys and moored sensors give a trustworthy number at one place. But "where is it unusually warm right now," "where might a bloom form next week," or "which stretch of coast are we sampling the least" are inherently spatial questions. Answering them needs either a live global grid (which is expensive to render well) or a model that can generalise from sparse labelled examples to the whole map — both of which are engineering problems most ocean-data tools never take on.

4. Forecast is usually a black box or a spreadsheet

Serious ocean forecasting exists (regional circulation models, reanalysis products), but it is either institutional-scale infrastructure or a raw number with no explanation of why the model

believes it, no comparison against a naive baseline, and no visible uncertainty. A forecast nobody can audit is a forecast nobody outside its own team should trust.

WHAT THIS ADDS UP TO

The ocean is one of the best-instrumented parts of the planet, and almost none of that instrumentation is usable by a non-specialist in real time. MarisAI's bet is that the fix is not more data — it's a platform that does the fetching, cleaning, aligning, modelling, and explaining, and hands back something a person can simply look at.

Who It's For

Audience	What they come for	Where in MarisAI
A marine researcher or student	Fast access to real, current ocean fields without provider-hopping; export for offline analysis	The map, the downloader, point brief/compare
A fisher or coastal operator	Where the water is currently favourable, where an eddy or upwelling front is sitting, what's forecast for the next few days	The map's flow layers, the Ocean Assistant, per-variable forecast pages
A policy analyst or NGO	Defensible, sourced, exportable evidence for a specific coastline or question — not a black-box number	Point brief/compare (PDF), the dashboard, biodiversity coverage
A teammate joining the project	An intuitive way to see what the platform actually does, without reading code	Everything in Part II of this guide
A hackathon judge	The idea, in two minutes, and evidence it's real and working — not a mockup	Chapters 1, 2, 4, 5
An engineer extending the platform	How the pieces fit, and the non-obvious decisions behind them, before changing anything	Part III and IV of this guide

What Makes It Different

It never fabricates a number

This is the single rule the whole codebase is built around: when data is missing, cold, or unavailable, the platform says so — it never substitutes a placeholder value that looks like a real reading. A blank chart is honest; a fake "0.00" is a false ocean measurement. Every panel, KPI, and layer in MarisAI carries an explicit availability flag and, when something can't be shown, a stated reason.

It shows its work

The AI assistant's answers are checked, after generation, against the raw tool results that produced them — any number in the reply that cannot be traced back to real data is flagged rather than trusted. Forecasts are scored against two honest baselines (naive persistence and a seasonal climatology), not just presented as "the model says." Detected features like ocean eddies ship with the statistical threshold that produced them, so a count is legible as a measurement under a stated rule rather than an unexplained fact.

It treats "forecast" as a discipline, not a demo

Models predict the *change* in a variable, not its raw level — a subtle choice that turned out to matter enormously: an early version of the forecasting engine that predicted absolute values scored *worse than simply guessing "no change"* at every time horizon. Retrained to predict deltas, the same data scored decisively better than that baseline. Every trained horizon has to clear a real bar — positive skill against persistence, and no more than one losing fold out of five — before it's allowed to ship, and failed candidates are kept, not deleted, as a paper trail for the decision.

It corroborates rather than asserts

Several of the map's more interesting layers are built from two independent signals rather than one. Coastal upwelling is detected from wind, then checked against sea-surface temperature — and the platform reports the agreement rate honestly (roughly 20% of favourable coastal water was actually cool, barely different from unfavourable water), rather than quietly dropping a check that came back weak. That kind of self-report is rare in dashboards generally and central to how MarisAI is built.

It is a real, deployed system — not a static prototype

Every field on the map is fetched live from its provider on a real schedule, cached, and refreshed; the forecasting engine trains real models against real historical data and serves real inference; the ML pipelines produce real, versioned model artifacts with tracked experiments. There is no "upload your API key here" step standing between a visitor and real ocean data — the credentials live server-side, and the browser never has to touch a NetCDF file.

A Two-Minute Tour

If you only have two minutes with someone — a judge, a new teammate, a visitor — this is the walk:

Minute one — the map

Open `/map`. Spin the globe. Turn on sea-surface temperature; the whole planet's current water temperature renders as a live colour field. Turn it off, turn on wind or currents; particles start streaming across the ocean in real time, animated from the actual live wind and current data. Turn on **Eddies** — every rotating current structure the platform's detector found right now appears as a ring sized to its real radius, coloured by rotation direction. Turn on a seven-day **forecast** layer for the same variable and watch the field change into what the model expects a week from now.

Minute two — ask it something

Open the **Ocean Assistant** and ask a real question about a coastline you care about — "is there an eddy near the coast of Chennai right now," or "what's the forecast heat-stress risk for the Andaman Sea this week." Watch it call the platform's own tools live, cite what it found, and flag anything it can't verify. Or click a point on the map and generate a **Point Brief** — a full one-page report on that exact coordinate, ready to export as a PDF.

THE LINE THAT LANDS WITH JUDGES

"Everything you just saw is live. That eddy ring, that forecast field, that answer the assistant gave — none of it is a screenshot or a canned demo path. It's the same live pipeline a researcher would get from the raw provider APIs, just made legible."

PART II

User Guide

A plain-language walkthrough of every page in MarisAI — what it shows, how to read it, and what each control does. No engineering background needed.

Getting Around the App

MarisAI has one navigation bar, present on every page, with links to the main sections: the **Map**, the **Dashboard**, the **Analytics** page, the **Downloader**, the **Assistant**, and **Feedback**. Two small controls live in the navbar on every page:

- **A theme toggle** — light or dark mode, remembered across visits.
- **A timezone selector** — every timestamp shown anywhere in the app (charts, briefs, chat) is displayed in whichever timezone you pick here, rather than a hardcoded UTC.

The map page is the one exception to the usual page-to-page transition — it is intentionally not animated in or out, because doing so would destroy and rebuild the 3D map engine every time you navigated to it. Everywhere else, pages cross-fade.

The Interactive Map

The map is MarisAI's centrepiece: a full-screen, spinnable, zoomable globe that layers live ocean data on top of a real basemap.

Basemaps

Four basemap choices sit in the map's style picker:

- **Abyss** and **Bathymetry** (the defaults) — smooth, continuously-zooming vector maps that redraw as geometry rather than a fixed image, so the zoom feels fluid at any level. Bathymetry additionally shades the seafloor to show underwater terrain.
- **Satellite** and **Dark Marine** — photographic-style raster basemaps. These blur briefly between zoom levels, the way a normal photo map does, because they are pre-rendered images rather than live geometry.

Layer categories

The layer panel groups everything into categories, shown as coloured tags throughout this guide:

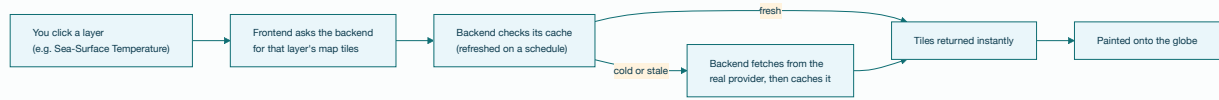
Category	What it means
OCEAN	A full-coverage colour field — temperature, salinity, chlorophyll, waves, and so on. Exactly one such layer paints at a time.
FLOW	Motion — animated wind and current particles, plus the eddy and upwelling detectors. Several flow layers can be stacked together.
AI (EXPERIMENTAL)	Seven-day forecasts and model-derived layers, clearly labelled as model output rather than direct observation.
EDNA	Where the ocean has actually been sampled for environmental DNA — a coverage map, not a species map.

Drawing an area and using the drift tool

The map includes a rectangle-draw tool for bounding a region (used by the downloader and by area-based queries), and a drift simulation that combines live currents, waves (Stokes drift), and wind into a single predicted drift path for an object at sea — with presets for different object types (a swamped hull drifts differently from a life raft, because wind pushes each a different

amount). The drift tool is explicit that it shows a single snapshot-based path, not a forecast with uncertainty bounds.

DIAGRAM 7.1 — WHAT HAPPENS WHEN YOU TURN ON A LAYER



Reading Map Layers, Plainly

Sea-surface temperature, wind, currents, waves, salinity, chlorophyll

These are the platform's core observed fields — read directly from satellite and reanalysis data, refreshed on a schedule appropriate to each (currents and wind refresh hourly; slower-changing biogeochemical fields refresh daily). Wind and currents render as moving particles rather than a static colour field, because direction matters as much as speed.

FLOW

Ocean Eddies

Every rotating current structure the platform can currently detect, drawn as a ring at its true physical size. **Teal rings turn one way (cyclonic), amber rings turn the other (anticyclonic)** — and which colour means which depends on which hemisphere you're looking at, because a vortex spinning the same direction is classified oppositely north and south of the equator. Tap a ring for its size, intensity, and rotation sense. This is a detector, not a tracker: each refresh is an independent snapshot, so an eddy does not carry an "age" — it either shows up in the current pass or it doesn't.

FLOW

Coastal Upwelling

Shows coastline stretches where the wind is currently pushing surface water offshore, which pulls cold, nutrient-rich deep water up to replace it — the process behind many of the world's most productive fishing grounds. Where satellite temperature data is available for the same coast, an outline shows whether the water is *actually* unusually cool right now, corroborating the wind-based signal. The platform is upfront that this corroboration is generally weak — worth checking, not worth over-trusting on its own.

EDNA

eDNA Sampling Coverage

This layer answers a different question from every other layer on the map: not "what is the water doing," but "has anyone ever molecularly sampled this water for species DNA." The honest answer, globally, is: almost nowhere. Sampled cells cover roughly the area of Belgium out of the entire ocean. The brighter a cell, the more DNA-based species detections have come from that spot — but that is a measure of research effort, not of how much life is actually there.

AI

Seven-Day Forecasts

For most variables that have a trained model, a forecast layer shows either the predicted field itself or the *change* from today — the change view is usually the more informative one, since the raw predicted field for next week often looks almost identical to today's. Patches of diagonal hatching mark water the model could not score (for example, where an input the model needs was itself unavailable) — hatching is deliberately different from "no data," which is shown as fully transparent.

The Ocean Intelligence Dashboard

The dashboard (`/dashboard`) is a single-screen overview: global KPI cards, live provider status, threshold-based alerts, and trend charts, refreshed on a schedule rather than on every page load.

KPI cards

Cards for global sea-surface temperature, coral bleaching risk, and heat-stress extent sit at the top. Each card is in one of three states, shown distinctly rather than left ambiguous: **ready** (a live number), **warming** (the server is still fetching the underlying data, shown as a progress indicator, not a blank), or **unavailable** (the source failed, with a stated reason). A cold start and a real failure never look the same.

Alerts

Rule-based flags over real fields — for example, a coastline crossing a heat-stress threshold. Each alert states the exact rule and value that triggered it. The panel is explicit that this is not an official marine warning service.

Trend charts

Historical charts for a wide set of variables, each honestly scoped to how far back that variable's own record actually goes — some reach back to 1940, others only to 2022 — rather than pretending every chart shares one history depth.

Per-Variable Metric Pages

Clicking through from the dashboard (or navigating to `/dashboard/<variable>`) opens a dedicated page for that one variable — sea-surface temperature, chlorophyll, wave height, and so on — with up to three sections, shown only when the variable actually supports them:

- **History** — an interactive, zoomable chart of the real recorded past.
- **Forecast** — the trained model's prediction, shown alongside its plotted history so the transition from observed to predicted is visible.
- **Explainability** — what the forecasting model is actually weighting to make its prediction (see Chapter 21).

A short "Ocean Story" at the top of each page turns the same numbers into a sentence or two of plain-language narrative — every number quoted in that narrative is checked against the real computed data before being shown; if a generated sentence claims a number that isn't backed by the underlying figures, it's replaced with a plain templated sentence instead.

The Universal Ocean Data Downloader

The downloader (`/download`) is MarisAI's export tool: pick a variable, a place (a point or a drawn area), a time range, and a depth if relevant, and get back a clean file — CSV, JSON, or a formatted PDF report — rather than a raw provider response.

What you can download

34 of the 36 variables in the platform's catalogue are available against real data today, spanning temperature, salinity, currents, waves, biogeochemistry (chlorophyll, oxygen, nitrate, pH, and more), wind, and seafloor depth, pulled from 14 different upstream providers. Two variables (tidal height and ammonium) remain unavailable — there is currently no global source for either.

Reading the result honestly

Every export states its provider, its native resolution, its coverage start date, and its licence — so a downloaded file is self-describing rather than an anonymous spreadsheet. Depth-resolved variables (temperature, salinity) let you pick a depth, snapped to the nearest level the underlying model actually has; a request deeper than the seafloor at that point correctly comes back empty, with an explanation rather than a silent zero.

Point Brief & Compare

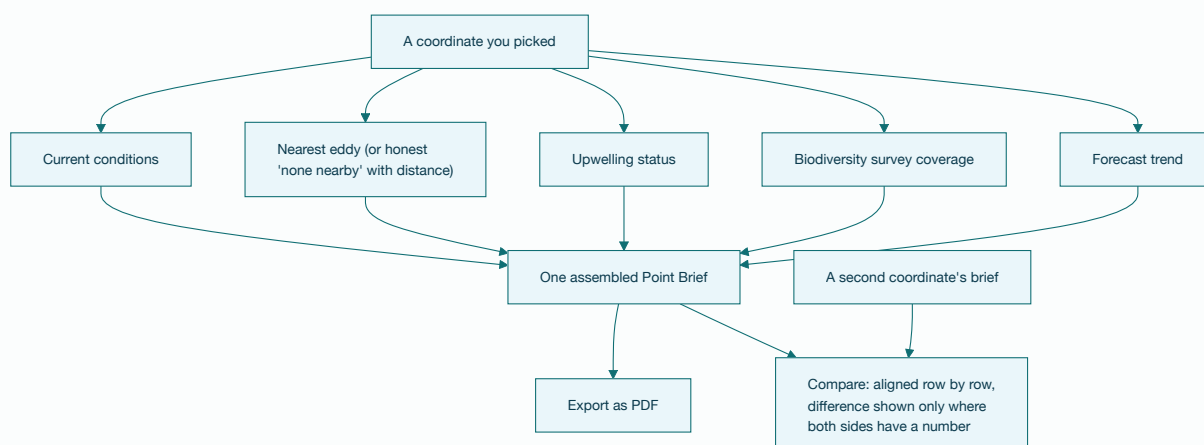
Point Brief

Click any coordinate on the map (or enter one directly) to generate a Point Brief — a single readable document assembling everything the platform knows about that spot: current conditions, nearby detected eddies (with the distance to the nearest one, even if none is close), upwelling status, biodiversity survey coverage, and forecast trend, exportable as a PDF. Every section states plainly whether it has data for that point and, if not, why — this is composition of real numbers the platform already computes elsewhere, not a separate calculation with room to disagree.

Compare

Compare (`/compare`) puts two Point Briefs side by side and aligns them row by row, computing a difference wherever both sides have a comparable number. A row that only one point has — a habitat suitability score inside the model's covered region and outside it at the other point, for instance — is shown rather than dropped, since that asymmetry is often the most useful part of the comparison. There is deliberately no single combined "score" ranking one point above the other; the platform has no independent way to validate a made-up composite metric, so it doesn't offer one.

DIAGRAM 12.1 — HOW A POINT BRIEF IS BUILT



The Ocean Assistant (AI Chat)

The Assistant (`/assistant`) is a conversational interface over the platform's own data and tools — you can ask it about a location, a phenomenon, or the platform itself, and it answers by calling real endpoints rather than by guessing from training data alone.

How to read an answer

Two things distinguish a MarisAI chat answer from an ordinary chatbot reply:

- A **"grounded" indicator** appears once the answer has finished streaming in. It reflects a real check: every number in the finished response is compared against everything the tools actually returned during that turn. A number the model invented or miscalculated — even a plausible one — gets flagged. This check can only run on a *finished* answer, so while text is still streaming in, treat it as provisional.
- **Tool activity is visible**, not hidden. You can see which of the platform's own services the assistant called to answer your question, so an answer about, say, an eddy near a coastline is traceably built from the same eddy detector described in Chapter 8, not a guess.

Ask it things like "what's the water temperature doing off this coast," "is there an upwelling event near here," "what does the platform mean by 'grounded,'" or "how do I read the eddy colours on the map" — it can answer questions about the ocean and about how to use MarisAI itself.

Feedback & Contact

The Feedback page sends a message directly to the team by email. It is a simple, honest channel — no ticketing system, no account required — for reporting something broken or suggesting an improvement.

PART III

Technical Architecture

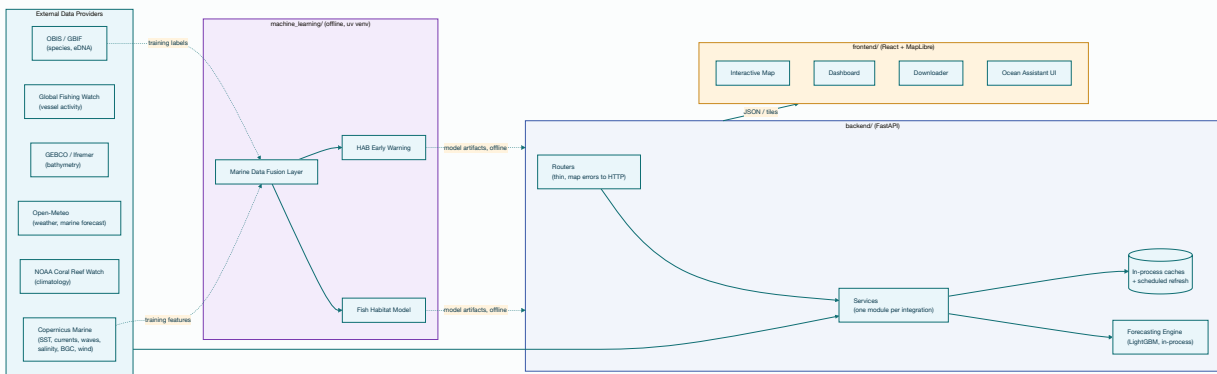
How MarisAI is actually built — the backend, the frontend, the detectors, the forecasting engine, and the machine-learning pipelines underneath. Written for engineers.

System Overview & Stack

MarisAI is three applications sharing one repository:

App	Stack	Role
backend/	FastAPI (Python), uv -managed	Proxies and processes every external data source, so credentials never reach the browser and NetCDF/GRIB scientific formats never need client-side handling.
frontend/	React 19, TypeScript, Vite, MapLibre GL JS 6, Zustand	The interactive map, dashboard, downloader, assistant UI, and every other page.
machine_learning/	Python, its own uv venv	Offline pipelines — harmful-algal-bloom early warning and fish habitat suitability — over a shared Marine Data Fusion Layer. Not imported by the backend at runtime; it produces model artifacts the backend later serves.

DIAGRAM 15.1 — SYSTEM OVERVIEW



The dotted arrows above are the one deliberate seam in the system: the ML pipelines produce model artifacts *offline*, and the backend later serves inference from those artifacts — `machine_learning/` is never imported by the running API process.

A real, migrated database schema exists (`core / observations`), but almost nothing queries it yet. Chat sessions are the one reference implementation of DB-backed persistence today. The operative question for a new feature is never "does this codebase use a database" — it does — but "does this feature produce records a user created and expects back later," versus configuration, which an in-code registry serves better.

Backend Conventions

Thin routers, real services

Routers catch a service-specific exception and map it to an `HTTPException` with a real status code — a raw provider exception or traceback never reaches the client. Services are plain async functions and modules, not classes, not database-backed. Each external integration is a self-contained module with its own error type.

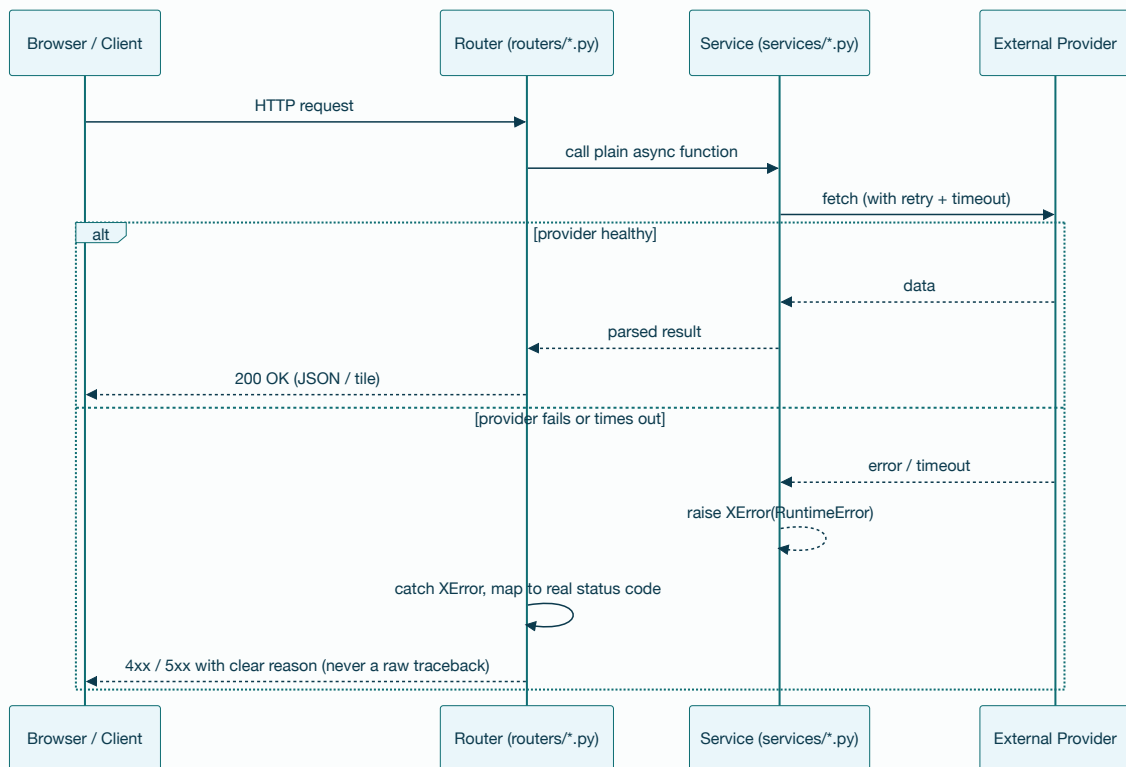
Scoping without accounts

Authentication was deliberately removed from this codebase. Where a feature needs to remember something for one person across visits (chat history), it scopes on a browser-generated UUID stored in `localStorage` — explicitly documented as convenience scoping, not access control. A feature that would send output to an address the caller supplies (email, webhook) is a different risk category entirely — that needs a confirmed address and a signed token independent of the UUID, because an unauthenticated, guessable-ID-scoped table that can be made to send mail is a spam relay.

Two Copernicus access patterns, and picking the wrong one is expensive

Copernicus Marine's zarr data has two services with opposite chunking. `arco-geo-series` (huge spatial chunks, one timestep each) is right for "whole globe, latest snapshot" — map-tile rendering. `arco-time-series` (fine spatial chunks, huge time chunks) is right for "one bounded area, many timesteps" — downloads and point forecasting. Using the wrong one for the access pattern turns a ten-second fetch into minutes or hours; this is re-verified empirically each time a new access pattern is added, never assumed.

DIAGRAM 16.1 — REQUEST LIFECYCLE: ROUTER → SERVICE → CLIENT



Near-real-time products can lag with placeholder data

Some Copernicus products publish a trailing timestep that exists in the index but is entirely empty — a "currently updating" placeholder. Code that wants "the latest" data walks backward and validates that data is actually present, not merely that a timestep exists.

Data Sources & Providers

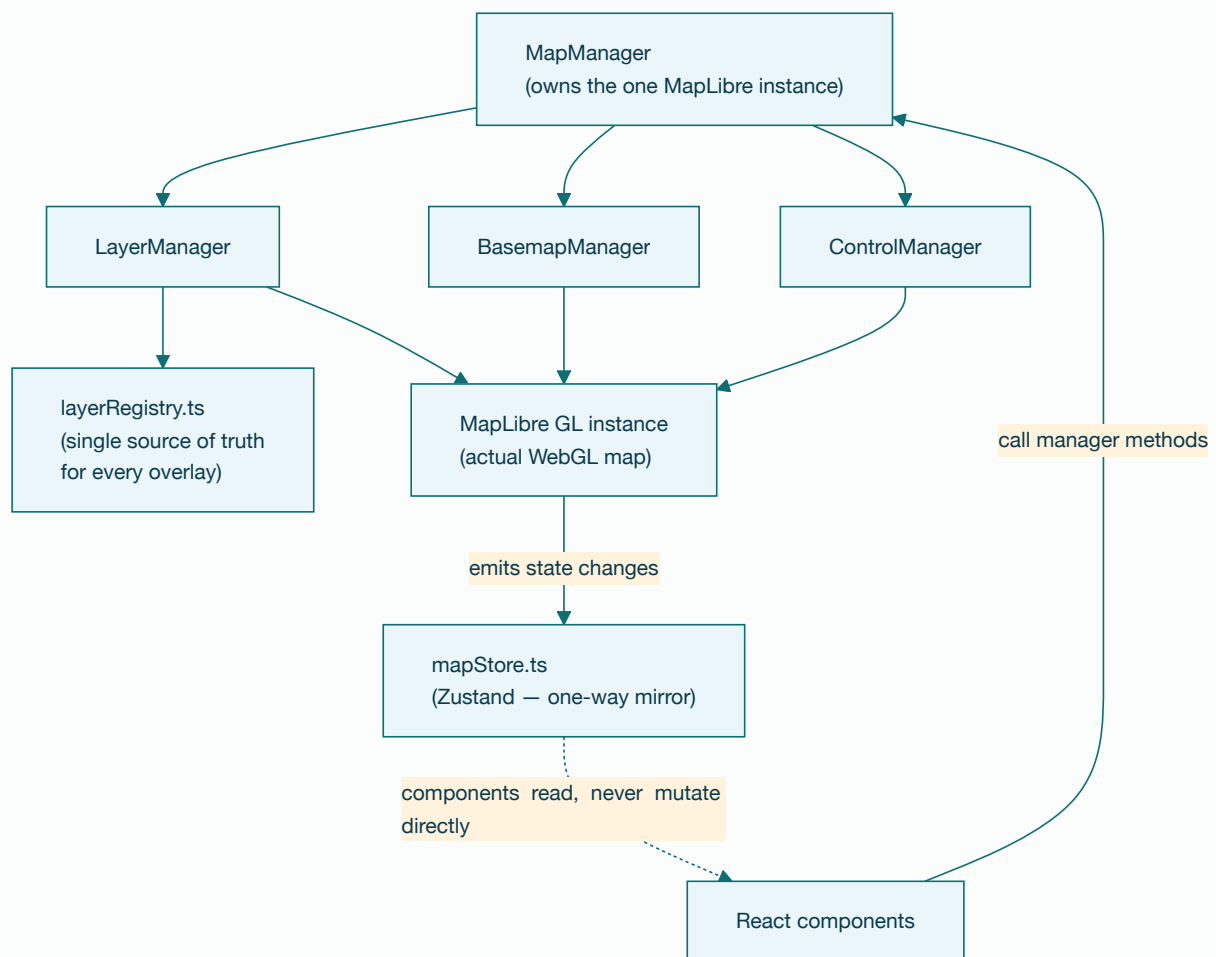
Provider	What it supplies
Copernicus Marine	Sea-surface temperature, currents, waves, salinity, biogeochemistry (chlorophyll, oxygen, nitrate, pH, and more), wind
Global Fishing Watch	AIS-based fishing vessel activity
GEBCO (via Ifremer ERDDAP)	Global seafloor bathymetry, 15 arc-second resolution
Open-Meteo	Weather and marine point forecasts
NOAA Coral Reef Watch	The platform's real climatology — the basis for bleaching risk, heat-stress extent, and SST anomaly
OBIS / GBIF	Species occurrence records, including DNA-sequenced (eDNA) detections, for biodiversity coverage and the ML pipelines' training labels

Every provider integration is expected to handle intermittent flapping gracefully. A finding that generalised across several of these hosts: ERDDAP-style servers answer `404` while a dataset is mid-reload, indistinguishably from a dataset that has actually been retired — so a retry policy that gives up permanently on a first `404` will occasionally "discover" that a perfectly live dataset is gone. The backend's retry logic (`history.is_retryable`) therefore retries `404` exactly once rather than zero or many times, alongside `5xx` and a few specific `4xx` codes that genuinely are retryable.

The Map Subsystem

The map (`features/map/`) is owned by a `MapManager` that wraps one MapLibre instance, with three child managers: `LayerManager`, `BasemapManager`, and `ControlManager`. A single `layerRegistry.ts` is the source of truth for every overlay layer, and `mapStore.ts` mirrors MapLibre's state one-way — components call manager methods, managers mutate the map and then emit back into the store, so the store never fights MapLibre for ownership of state.

DIAGRAM 18.1 — MAP MANAGER COMPOSITION



Basemaps are a discriminated union

Raster basemaps (Satellite, Blue Marble, Dark Marine) are one source, one layer — a fixed tile image that blurs and pops between zoom levels, since zoom is simulated by scaling a bitmap

until the next tile loads. Vector basemaps (Abyss, Bathymetry) are many sources and roughly thirteen to fourteen GPU-rendered style layers, redrawn every frame at the exact fractional zoom — which is what makes their zoom feel continuous rather than stepped.

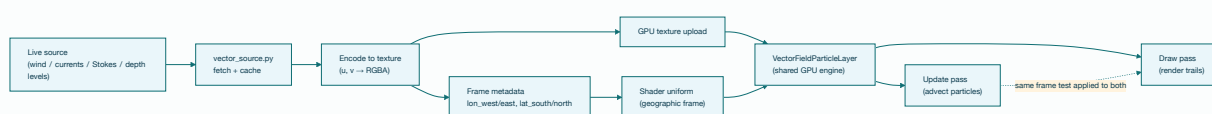
Hazards specific to this stack, worth knowing before touching it

- The underlying vector tile set (OpenMapTiles) has no landmass polygon — land is the background colour and water is drawn on top of it. Building the style the "obvious" way (an ocean background with a land fill layer) renders a nearly all-blue planet.
- Only two font weights are actually served by the glyph endpoint. Requesting a missing one makes MapLibre silently drop the entire text layer with no error.
- Calling `map.setGlyphs()` mid-session triggers an async style reload that can silently drop sources and layers added in the same tick — glyphs are therefore set once, at initial style construction, and never changed at runtime.
- Basemap source specs are shared, module-level objects (because both the full-screen map and the dashboard's embedded map panel can be alive simultaneously) and must be deep-cloned before being handed to `addSource` — MapLibre mutates the object it's given, so passing the shared original lets one map instance corrupt state another is still using.
- There is deliberately no 3D terrain. It was built, then removed: MapLibre drapes raster overlays onto the terrain mesh, and on a bathymetric elevation model that mesh is the seafloor — so a sea-surface temperature layer with realistic terrain exaggeration ended up rendered kilometres below the camera. Sea-surface data and raised seafloor geometry are mutually exclusive on this stack; the bathymetric elevation data survives only as flat hillshading, which doesn't displace geometry.

Vector-Field Particle Engine

One GPU particle engine drives every animated flow layer — live wind, live surface currents, Stokes (wave) drift, currents at six depth levels, a combined drift field for objects at sea, and forecast currents at each horizon. What generalises across all of them lives in `services/vector_source.py` (fetch, cache, encode, point-sample) behind a common `VectorSourceSpec`.

DIAGRAM 19.1 — VECTOR FIELD PIPELINE (FETCH → GPU PARTICLES)

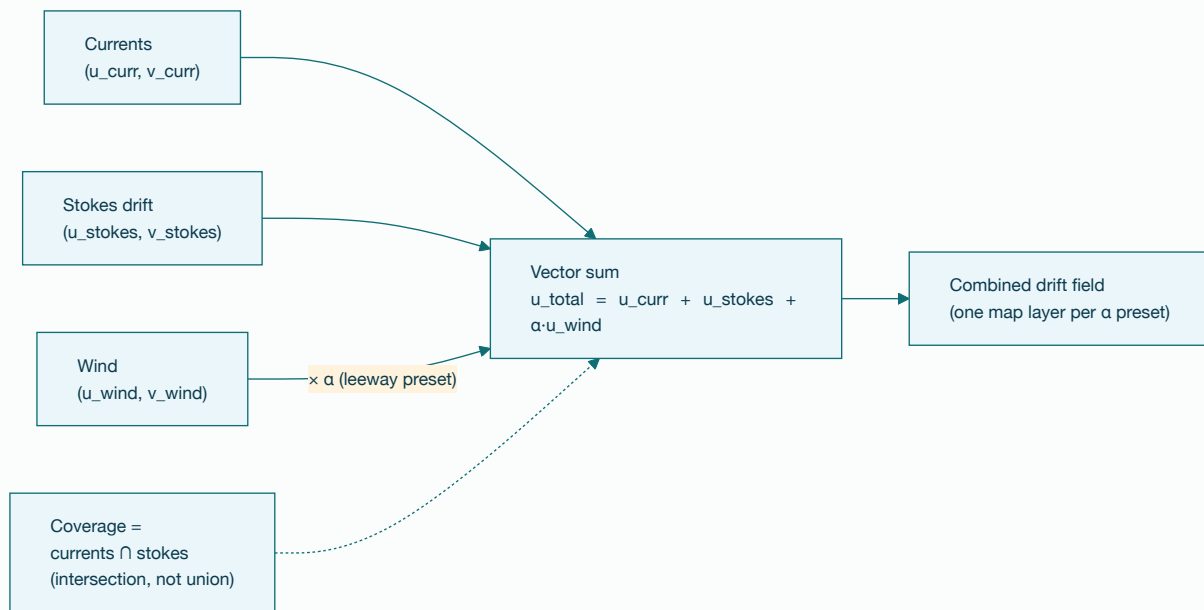


Composing fields correctly

A drifting object's combined field is a *sum of components*, never a sum of bearings: $u_{total} = u_{currents} + u_{stokes} + \alpha \cdot u_{wind}$. The wind multiplier, `alpha` (leeway), belongs to the object being simulated, not the ocean — a swamped hull picks up roughly 1.5% of wind speed, an undrogued life raft roughly 6% — so it is a request parameter with named presets, not a constant baked into the field.

Coverage is the *intersection* of the two water terms (currents and Stokes drift), not their union — treating a cell the wave product doesn't cover as zero drift would understate exactly the high-sea-state water where Stokes drift matters most. Wind is deliberately excluded from the coverage check, since it's defined over land too and only ever adds to a cell that already has water in it.

DIAGRAM 19.2 — COMBINED DRIFT FIELD (SUM OF COMPONENTS)



The texture/shader contract

Each field's geographic frame — its literal outer edges in latitude and longitude — is passed to the shader as data, not assumed as a constant. Copernicus's global current grid runs from -80° to 90° latitude, not the $-90^\circ/90^\circ$ a naive formula would assume; reading it with the wrong frame stretches sampling and advects particles with water from the wrong place while still animating convincingly. Because this contract crosses a Python/JavaScript boundary that nothing type-checks automatically, a test reimplements the shader's coordinate mapping in Python and asserts the round trip is exact.

Direction conventions are not interchangeable

Currents are named for where the water is going; wind, by long meteorological convention, is named for where it's *coming from* — 180 degrees apart. Reusing one field's direction formula for the other draws every arrow backwards, and it still looks entirely plausible on screen. Circular quantities like current direction are handled from the model all the way to the screen as sine/cosine pairs recombined with `atan2`, never as a bare degrees value that a linear model or a linear colour ramp would treat 359° and 1° as maximally different rather than adjacent.

Detectors: Eddies, Upwelling, eDNA

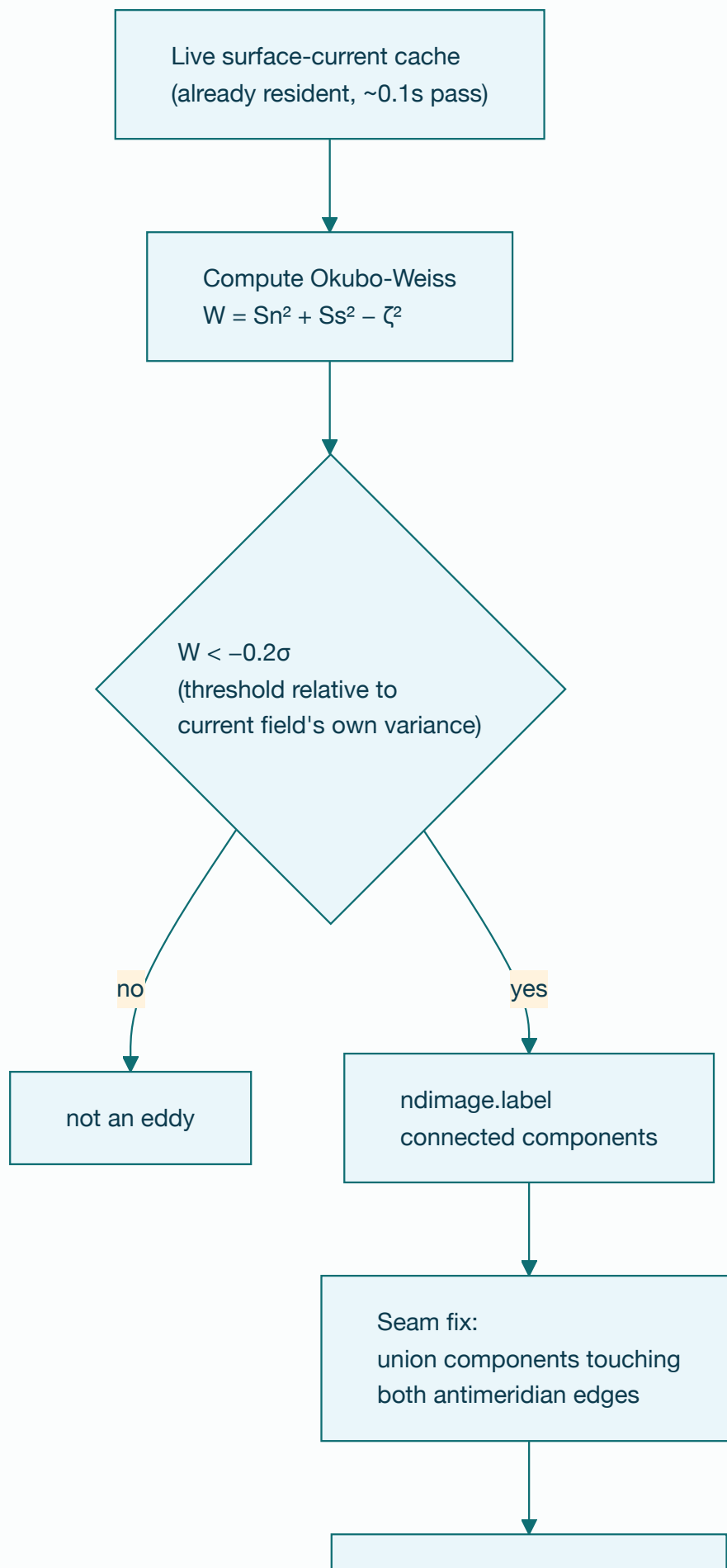
Eddy detection

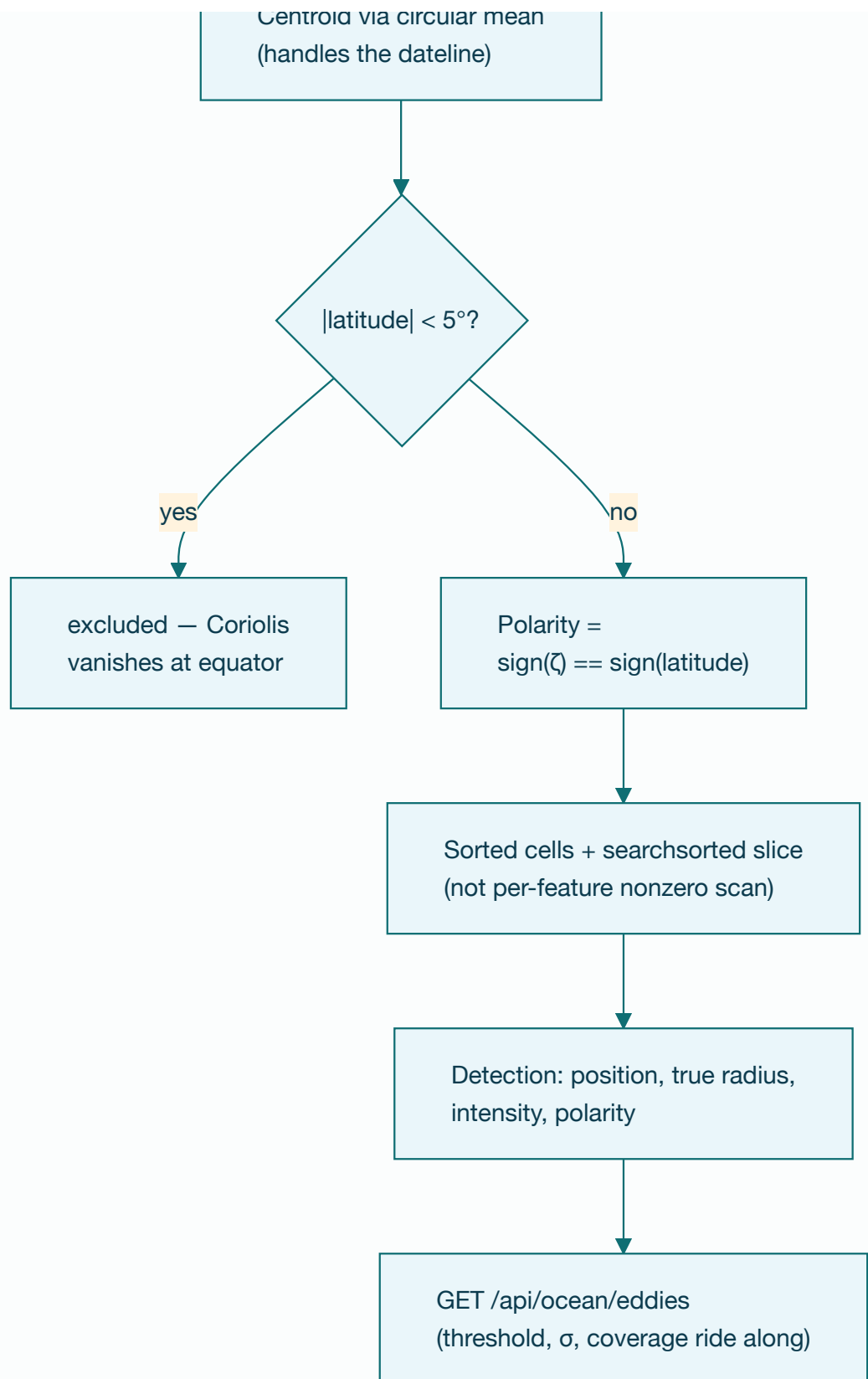
MarisAI's first true *detector* — it turns a continuous field into named, located, sized things. The method is the Okubo-Weiss parameter over the live surface-current cache: a cell is flagged where rotation (vorticity) dominates strain, relative to the variance of the field currently in view. Two properties matter for correct interpretation:

- **It detects; it does not track.** Nothing holds state between refreshes. Assigning an identity to the "same" eddy across frames is a genuinely hard matching problem, and a matcher that flickers produces tracks that are algorithmic artefacts presented as observations — so no eddy here carries an age or a trajectory, deliberately.
- **Polarity depends on hemisphere, not just rotation direction.** Cyclonic means rotating with the planet's own spin, so an identical counter-clockwise vortex is cyclonic north of the equator and anticyclonic south of it. A narrow band around the equator is excluded entirely, since the Coriolis effect the classification depends on vanishes there.
- **The threshold is relative, not absolute** — a consistent detector under a changing view, not a census. The count therefore ships alongside the threshold and variance that produced it, so it's legible as a measurement under a stated rule.

DIAGRAM 20.1 — EDDY DETECTION PIPELINE



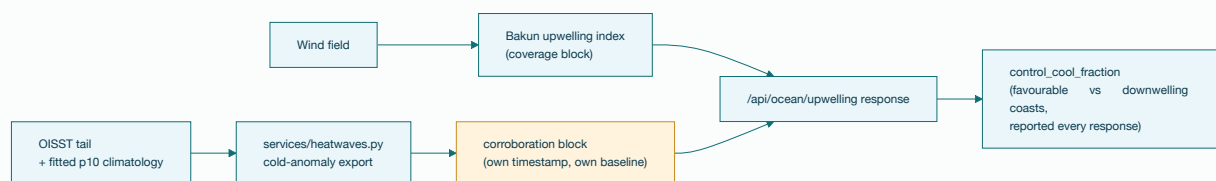




Coastal upwelling

Bakun's wind-derived upwelling index, corroborated where possible against real sea-surface temperature data from the same coast. The two claims are kept structurally separate — the wind index stands on its own and stays fully readable with no SST data at all, and the SST corroboration adds a second claim rather than filtering or editing the first. A cold SST cache degrades the layer to "wind signal only," never to nothing. The platform also reports its own control comparison: favourable-for-upwelling coastal water was measured only marginally more likely to actually be cool than unfavourable water — a genuinely weak agreement, stated rather than hidden, alongside every response.

DIAGRAM 20.2 — UPWELLING: TWO INDEPENDENT CLAIMS



The index (left) is complete and readable on its own; corroboration (right) only ever adds a claim, never edits or filters the index — a cold SST cache degrades the layer, it never breaks it.

eDNA sampling coverage

Not a new data integration — a new *question* asked of data the platform's biodiversity endpoint already had, filtered to DNA-derived detections. The headline finding the layer is built to communicate: global molecular ocean sampling covers roughly 0.0075% of the ocean at a meaningful cell size — smaller than Belgium — so the empty map is itself the finding, not a loading failure. Coverage percentages are always quoted at one fixed reference cell size, because the same records can be made to look like anywhere from under 1% to over 20% coverage purely by choosing a coarser grid cell — a coarse cell credits a single water sample with everything around it.

The Forecasting Engine

One framework predicts every forecastable variable — not one bespoke model per variable. Adding a new one is a single configuration block plus a training run, not new code.

Predict the change, not the level

The single most consequential design decision in the engine: models are trained to predict the *change* from the latest observation, not the raw future value. A tree-based model cannot represent "the answer equals the input" exactly, so on these highly autocorrelated ocean series, a level-fitted model actually scored *worse than naive persistence at every horizon, including one day out*. Retrained on the delta with everything else unchanged, the same data scored decisively better than persistence. This also means a missing final observation is a hard error at inference time, not a guess — there is nothing to add the predicted delta to.

DIAGRAM 21.1 — DELTA-TARGET TRAINING, END TO END



Syntax error in text
mermaid version 11.17.2

DIAGRAM 21.2 — WHY DELTA BEATS LEVEL



Delta-fitted model

$$\text{target} = y(t+h) - y(t)$$

persistence becomes
the trivial constant 0

skill = +0.20
(same data, same features)

Level-fitted model

target = $y(t+h)$

tree cannot represent
 $y(t+h) = y(t)$ exactly

skill = -0.12
(worse than persistence
at every horizon)

Two honest baselines, not one

Every model is scored against naive persistence *and* against a day-of-year climatology fitted with a circular window (never a bare single-day average, which on these record lengths would be an average of about two samples). Skill against persistence tends to rise with forecast horizon on several variables — which sounds good until you notice that is exactly the signature of a model that has only learned the seasonal cycle, since persistence decays with horizon and a seasonal cycle doesn't. A follow-up feature-ablation study, written up separately, found this reading was itself too quick: removing every calendar feature from most of the affected models left their skill against persistence essentially unchanged — so losing to climatology mostly reports how strong that baseline is, not what mechanism the model actually learned. Two baselines alone cannot distinguish those explanations; only the ablation can.

A real shipping bar

A trained horizon ships only if it beats persistence *and* loses on no more than one of five cross-validation folds — applied by a separate script from the trainer itself, deliberately, because the trainer has no concept of rejection and would otherwise silently resurrect a horizon that had already been deleted on its own merits during a routine retrain. Rejected models are moved to a dated rejected-models folder rather than deleted — the artifact is the evidence for the decision, kept rather than erased.

Provider reliability inside a training run

The single worst failure mode found in this engine was not an error at all: an upstream Copernicus read that simply never returned, silently occupying a training run for over ninety minutes with nothing ever raising an exception. Every provider fetch is now individually time-bounded and retried per-provider rather than as a whole batch, so a slow read from one source can't force a fast, healthy source to be retried unnecessarily alongside it.

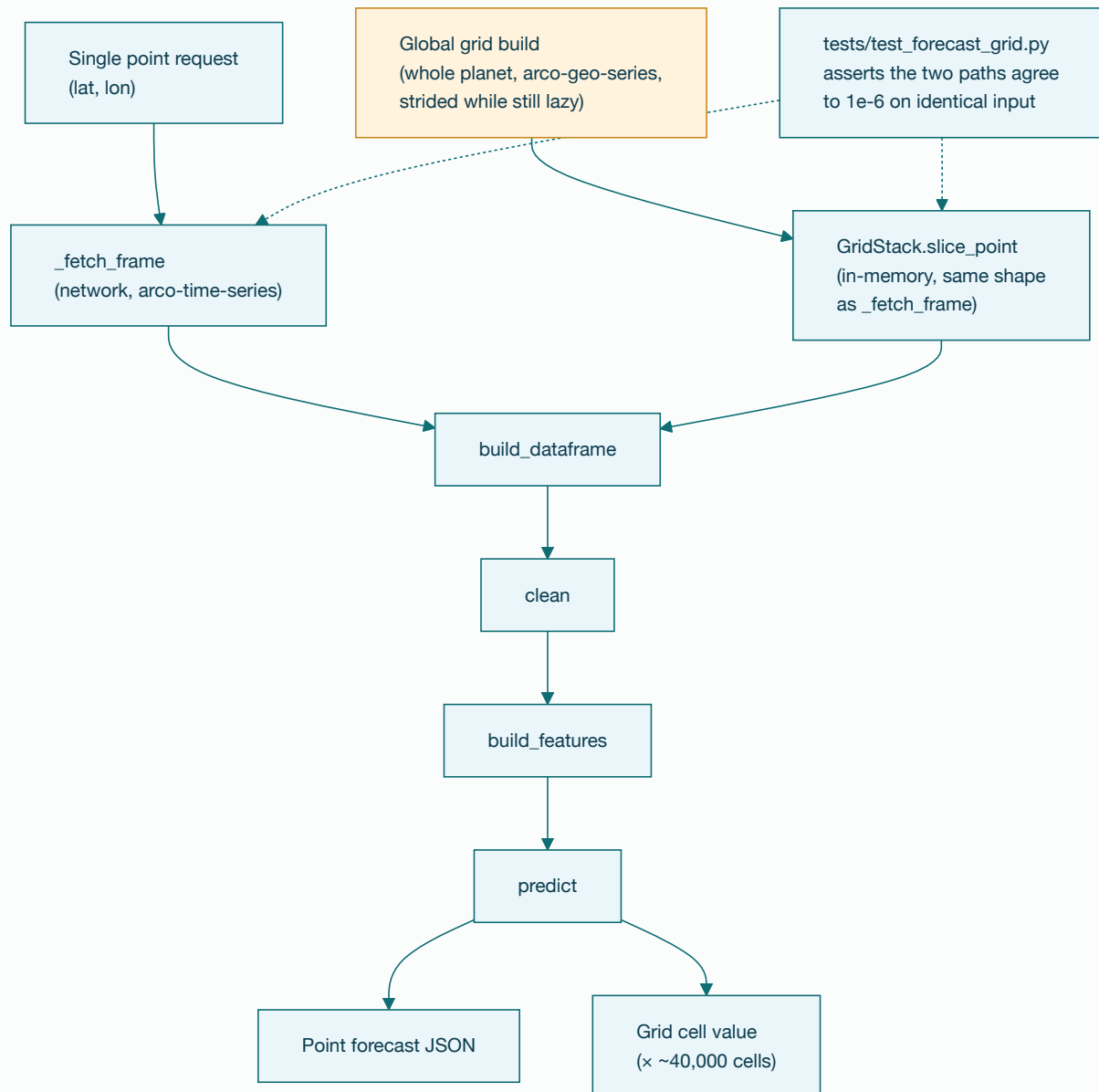
The Forecast Map (Global Grids)

The forecasting engine answers one (variable, point, horizon) question at a time. The forecast map renders the same engine as a full global colour field by inverting the access pattern: instead of one point over many timesteps, it fetches the whole globe, once per timestep, using Copernicus's geo-series access pattern designed for exactly that shape of request.

One code path, not two

The single most important correctness property of this feature: the grid path reuses the exact same clean → feature-build → predict pipeline the single-point forecast uses, cell by cell, rather than a separate grid-optimised implementation. A test compares the two paths on identical synthetic input and asserts they agree to within a tiny numerical tolerance — which is what makes it safe to optimise the tens-of-thousands-of-cells loop at all, since any shortcut that changed the answer would be caught immediately rather than discovered as silent drift between what a point forecast says and what the map shows for the same spot.

DIAGRAM 22.1 — ONE PIPELINE, TWO ENTRY POINTS



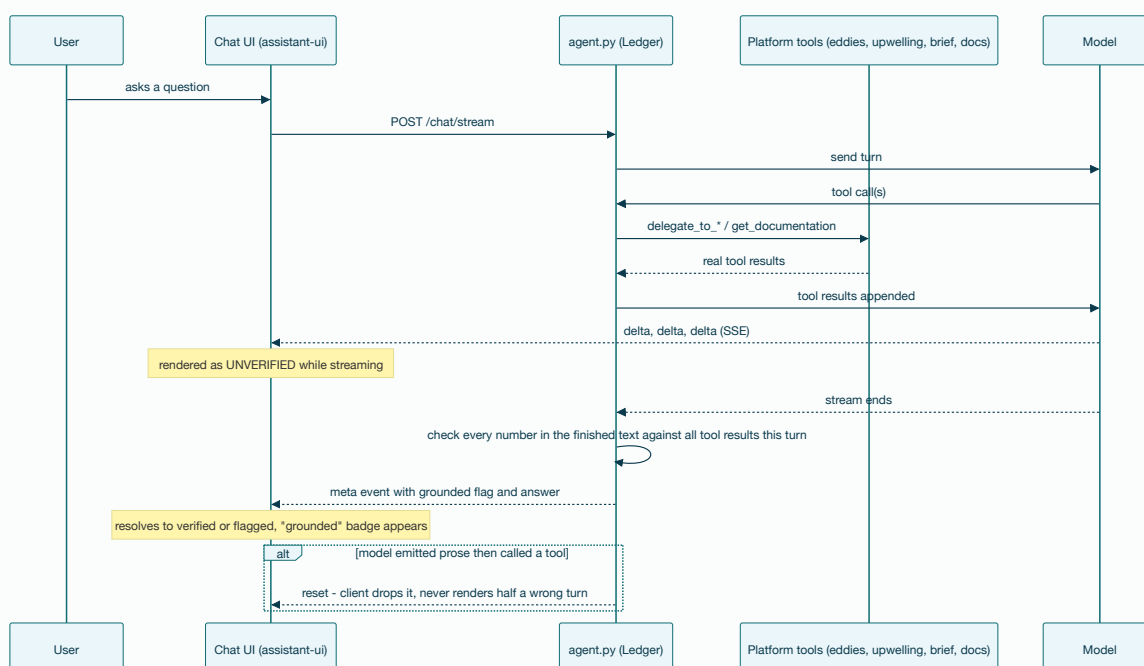
Rendering unforecastable water honestly

Where a cell has a real current observation but the model couldn't score it (a needed input was itself missing), the tile is drawn with diagonal hatching rather than either a fabricated colour or plain transparency — transparency is reserved for land and genuinely out-of-coverage water. Every colour ramp on this layer was measured for contrast against the map's near-black default basemap and adjusted so the darkest step of the ramp stays visibly distinguishable from open water, rather than the two blending together at typical layer opacity.

The Ocean Assistant, Architecturally

A bounded tool-calling loop over the platform's own services, served two ways from the same underlying logic: a plain JSON endpoint that returns a whole turn at once, and a server-sent-events streaming endpoint. Neither supersedes the other.

DIAGRAM 23.1 — STREAMING CHAT TURN WITH GROUNDING CHECK



Why grounding can't stream

The "grounded" check works by comparing the model's *finished* text against everything every tool call returned during that turn — it is therefore only knowable after the last token has arrived, and rides on a terminal event once streaming ends. The interface is built around this constraint deliberately: streamed text renders as unverified while it's still arriving, and only resolves to a verified/flagged state once the turn is complete.

Grounding's number check handles real formatting

The regular expression that scans an answer for figures it can't trace back to a tool result understands thousands separators. Without that, a depth quoted as "2,048 m" would split into "2" and "048," neither of which appears verbatim in any tool result, and a completely correct

answer would be wrongly flagged on every measurement over a thousand of something. A banner that fires on correct answers trains people to ignore the one time it matters.

Self-knowledge is a tool, not a fourth specialist

Questions about the platform itself ("what does 'grounded' mean," "how do I read the eddy colours") are answered from a static documentation index, not a live measurement — so this capability is bound directly as a top-level tool alongside the delegate-to-specialist tools, rather than routed through the specialist-orchestration path meant for live data questions. The documentation index is exported from the same in-app docs content used by the platform's own documentation search, so there is exactly one place that content is written and no second copy to drift out of sync.

The Universal Downloader, Architecturally

Two registries, deliberately kept separate: one maps each of the 34 variables to a provider and source field; the other maps each of the 14 providers to its dataset, native grid spacing, cadence, coverage start, citation, and licence. The service layer is a generic loop over the provider catalogue — adding a new provider is one catalogue entry, not an edit spread across several files.

DIAGRAM 24.1 — REGISTRY / CATALOG SPLIT

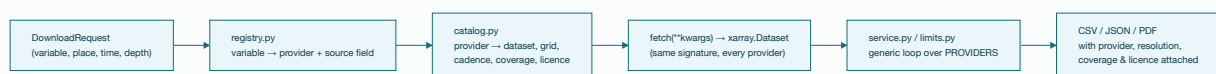
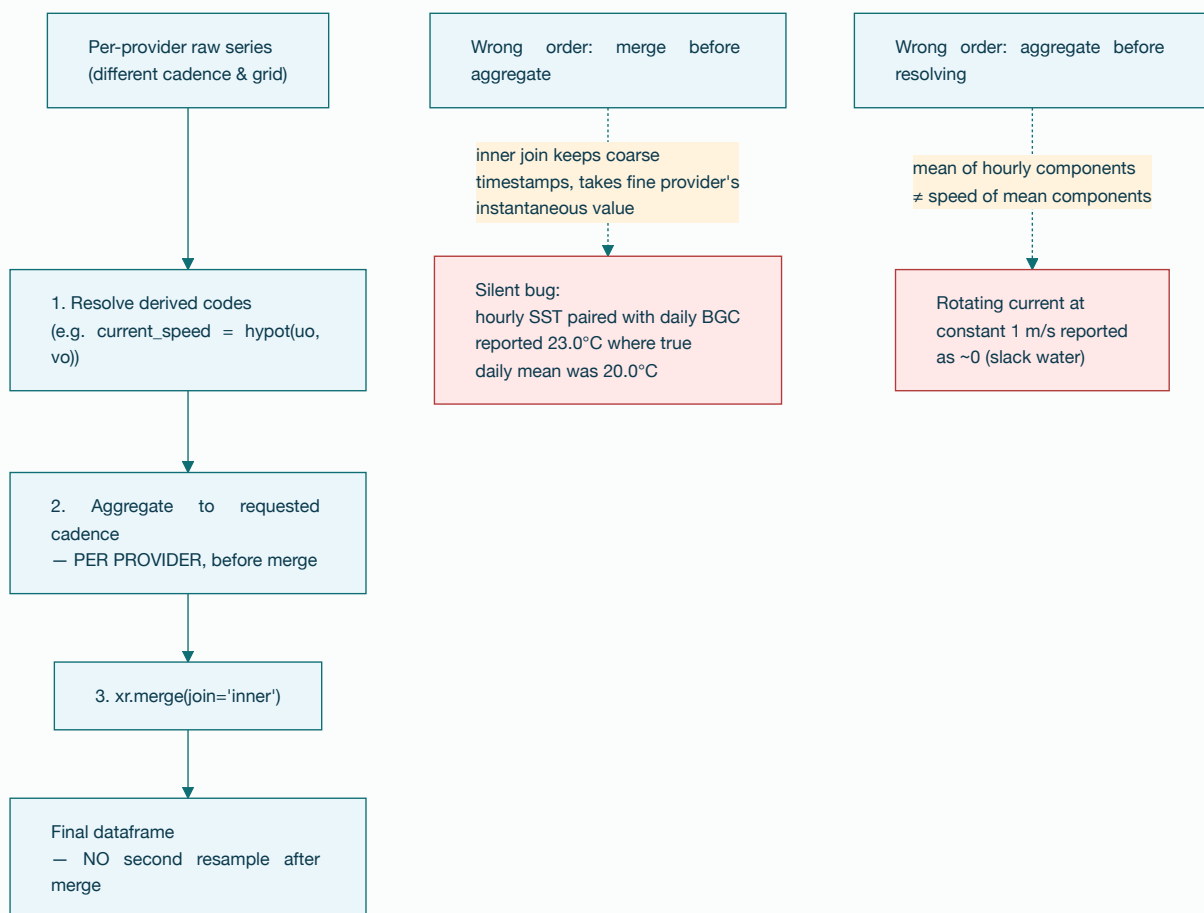


DIAGRAM 24.2 — WHY ORDER MATTERS: RESOLVE → AGGREGATE → MERGE



Aggregate before merging, and resolve derived fields before aggregating

Two ordering rules govern how multiple providers at different cadences and resolutions are combined into one dataframe, and both were earned by a real, previously-silent bug:

- **Time is aggregated to the requested cadence per provider, before an inner join across providers.** Left to itself, an inner join keeps the coarser provider's timestamps and pairs them with the finer provider's *instantaneous* value at that moment — an hourly field paired against a daily one silently reports a single midnight sample standing in for what should have been a full-day mean. On a synthetic test case this produced a value wrong by the full diurnal swing, in the same direction every time, with nothing raising and nothing NaN. Over a dozen of the platform's forecasting variables were feeding at least one covariate through this bug before the ordering was fixed.
- **Any derived quantity (like current speed from its east/north components) is resolved before aggregation, not after,** because aggregation and derivation don't commute for a non-linear formula — a current rotating steadily through a day at constant speed averages its raw components to roughly zero, so averaging first and computing speed second would report a rotating current as slack water.

There is deliberately no second resampling step after the merge — one place defines the requested cadence, not two, since two places that could define it is exactly how a cadence bug like the above stops being noticed.

The Dashboard Backend

Every dashboard data source is cached and refreshed on a schedule rather than fetched per request — the one exception being trend charts, which are inherently specific to whatever point or range is being asked about. The previous successful fetch is always kept on a failed refresh, so a transient upstream outage degrades the displayed data's freshness rather than blanking the page.

DIAGRAM 25.1 — CACHE-AND-REFRESH CYCLE



Syntax error in text
mermaid version 11.17.2

A real climatology, carefully masked

NOAA Coral Reef Watch is the platform's only true long-run climatology, and it's what makes bleaching risk, heat-stress extent, and SST anomaly possible at all. Three geographic maskings on top of it were each found by a number that didn't pass a sanity check: unmasked polar cells near the sea-ice margin produced double-digit anomalies that tripled the reported global mean; unmasked high-latitude cells produced a reported bleaching-risk maximum from an estuary and an inland sea, nowhere near an actual coral reef; and a raw temperature-anomaly threshold flagged nearly half the ocean as "heat-stressed," where the field's own accumulation-based criterion correctly flags roughly an eighth and reports zero across the winter Southern Ocean.

Ocean heat content, and why the reference depth is the wrong one to lead with

Heat content is computed, not fetched, and shipped at four integration depths rather than one. The conventional climate-reference depth (0–700m) is deliberately not the headline number for most marine questions — what drives a cyclone or shows up as a marine heatwave lives in roughly the top 100 metres, and six hundred metres of nearly constant deep water underneath dilutes that signal by more than 3x in the measured comparison. The shallower layers were

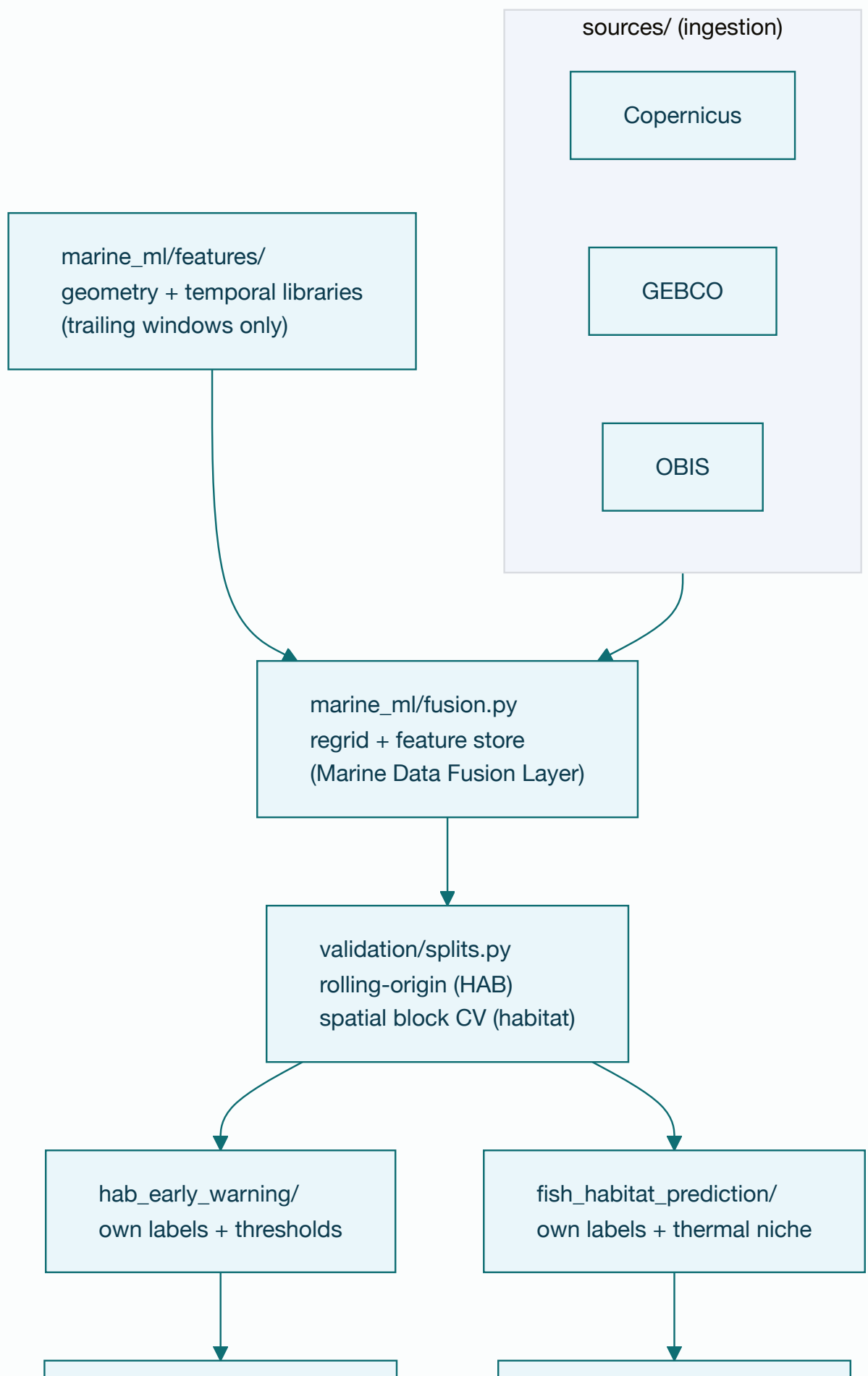
added rather than substituted, since the deep reference number remains the standard one for climate comparisons even though it under-represents what most users of this dashboard actually care about.

Offline Machine Learning Pipelines

Two problems — harmful algal bloom (HAB) early warning and fish habitat suitability — share one spine, `marine_ml/`: data ingestion, a regridding and feature-store layer (the Marine Data Fusion Layer), shared geometry and temporal feature libraries, and a validation harness. Only labels and problem-specific derived features live in each problem's own directory.

DIAGRAM 26.1 — THE SHARED SPINE





Model artifact
+ tracking.py run record

Model artifact
+ tracking.py run record

Leakage discipline is structural, not a checklist

- Every fit-then-apply pair (a climatology, a bloom threshold, a thermal niche) is deliberately split into two functions, so fitting on the wrong rows — including the test period — takes real, deliberate effort rather than being the path of least resistance.
- Exactly one forward-looking time shift exists anywhere in this half of the codebase — the HAB target construction itself — and a test scans the source to keep it that way; every other temporal feature is trailing by construction.
- Cross-validation is never random. Rolling-origin splits with an embargo gap for the bloom model, spatial block cross-validation for the habitat model — a random split on either problem measurably inflates every metric, because nearby rows in space or time are not independent samples.

DIAGRAM 26.2 — FIT/APPLY SPLIT PREVENTS LEAKAGE

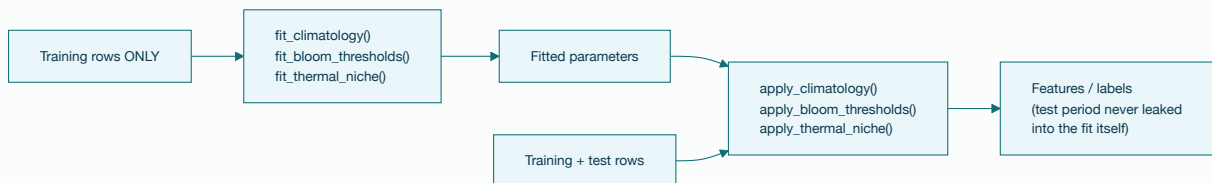
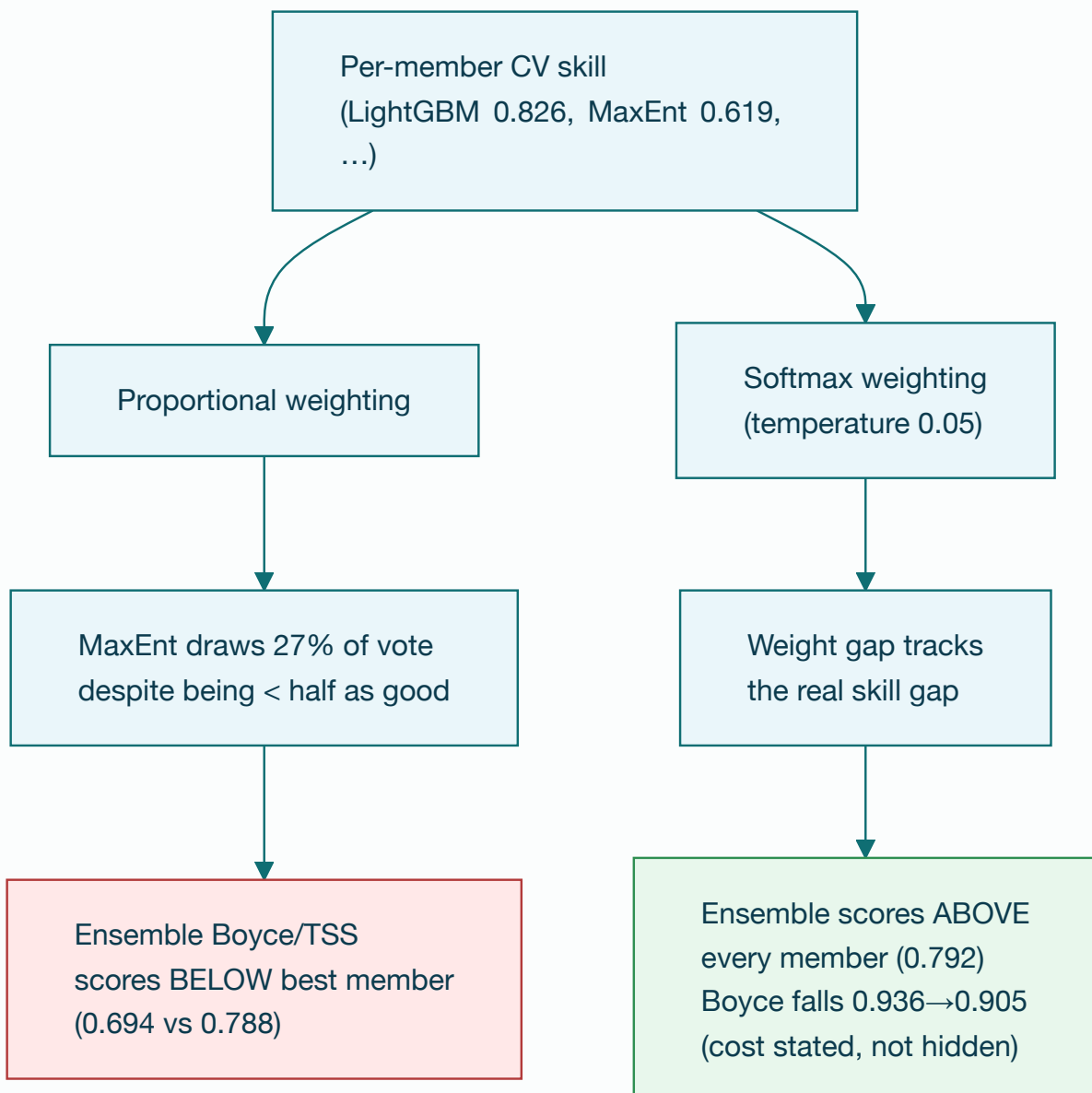


DIAGRAM 26.3 — ENSEMBLE WEIGHTING: SOFTMAX VS PROPORTIONAL



Regional beats global, and the arithmetic explains why

A worldwide version of the habitat model was built and directly measured against the regional one on identical held-out water: the regional model scored decisively better on every metric, despite training on roughly sixty times fewer rows. The regional model remains the served artifact for this reason. Similarly, a fully global version of the bloom-detection pipeline was costed out rather than assumed impractical: the same six-year window worldwide would be on the order of six hundred gigabytes, and mostly open-ocean rows that are near-constant

negatives, since blooms are specifically a coastal and upwelling phenomenon — confirming that multi-region, not global, is the right shape for that problem.

An ensemble is a decision, weighted by evidence

The habitat model's exported prediction is an ensemble across model types, and how the members are weighted is not a cosmetic choice: weighting proportionally to raw cross-validation skill compresses a large real quality gap into a small weight gap, because the skill metric floors at zero — so a member roughly half as good as the best one can still draw over a quarter of the vote. Switching to a softmax over the same skill scores gave an ensemble that finally scored above every individual member, at a stated, visible cost to spatial calibration versus the older weighting scheme — the tradeoff is recorded, not hidden.

Experiment tracking, without a modelling dependency in the backend

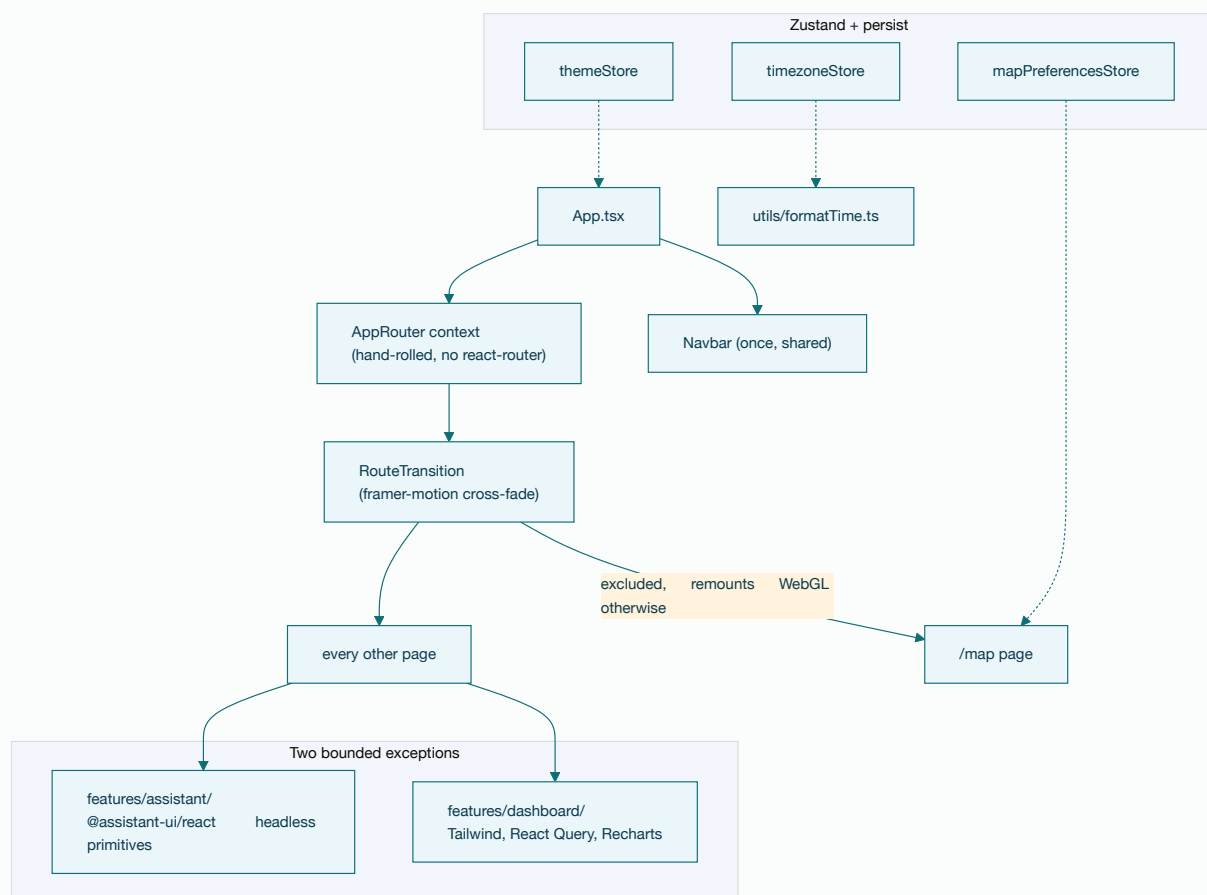
Every pipeline run is logged rather than overwriting the same output file on every rerun, so results accumulate instead of erasing their own history. The production backend's forecasting engine is tracked without importing the tracking library at all — it writes a plain, immutable JSON record per run, and the tracking side reads that record from outside the backend's own import graph. A backend test asserts the tracking library's name never appears anywhere in backend code, specifically to stop the natural-looking "improvement" of logging directly from the training script, which would reintroduce exactly the dependency this design avoids. Tracking is also built to never fail a training run — every logging call degrades to a warning rather than an exception, since a training run can take the better part of an hour and losing it to a logging failure would be absurd.

Frontend Architecture

Routing and page composition

Routing is hand-rolled rather than a routing library — a small context plus a plain if/else on the current path, with one shared navigation bar rendered once above it. Page transitions cross-fade everywhere except the map route, which is deliberately excluded: animating it in with a keyed wrapper would force React to remount the map component on every navigation, destroying and rebuilding the entire WebGL context and discarding the layer state a user had set up.

DIAGRAM 27.1 — FRONTEND COMPOSITION



Global preferences and per-page theming

Cross-page preferences (theme, timezone) live in small persisted state stores, backed by `localStorage`. Per-page visual theming follows the opposite pattern deliberately: each

themeable page defines its own small set of named colour variables rather than every page sharing one global palette directly, aliased to one underlying token file so there remains exactly one place a colour or type scale is actually chosen, even though each page still reads its own locally-named variables. The initial theme is stamped before the page's own script runs, specifically to avoid a visible flash from the default theme to the stored preference on first paint.

Two deliberate exceptions to a hand-rolled, dependency-light frontend

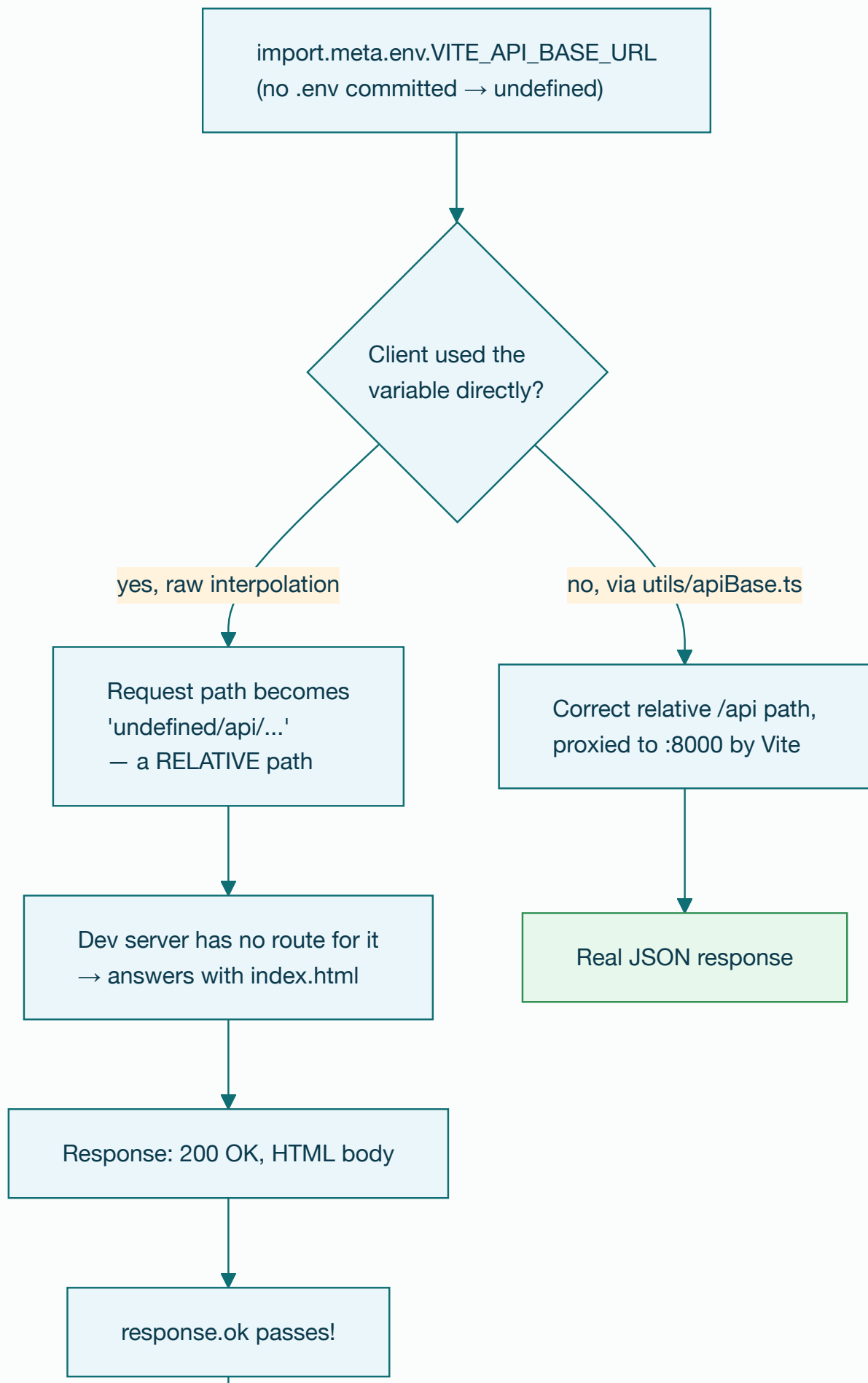
Nearly the whole frontend avoids UI kits, CSS frameworks, and form/validation libraries by convention — everything is hand-rolled CSS and plain components. Two feature areas are explicit, bounded exceptions: the dashboard, which uses Tailwind (with the global element reset intentionally disabled, since it would otherwise restyle every hand-CSS page in the app), React Query, and Recharts; and the assistant chat thread, which uses only the headless primitives of a chat UI library, styled with the app's own existing CSS rather than that library's own component styling.

API clients follow one shape and one base-URL rule

Every API client lives under one folder and follows the same shape: build a URL, fetch, throw a real error on a non-OK response. Critically, the backend's base URL is always read through one small helper rather than interpolating the environment variable directly in each client — because in an ordinary local checkout that variable is undefined, the resulting request path silently becomes a relative one, the dev server answers any unmatched relative path with its own HTML shell, and the fetch consequently returns a *200 OK response containing an HTML page* rather than an error. That passes every status check and only fails deep inside JSON parsing, so it manifests as unrelated-looking symptoms rather than a clean, obvious bug.

DIAGRAM 27.2 — THE UNDEFINED-BASE-URL TRAP







Fails deep inside .json()
'Unexpected token < in JSON'

PART IV

Setup, Development & Operations

Running MarisAI on your own machine, configuring credentials, and what's intentionally left for later.

Running MarisAI Locally

Backend

Dependencies are managed with `uv` (`pyproject.toml` + `uv.lock`). A generated `requirements.txt` also exists for non-`uv` installs; it must be regenerated after any dependency change with `NO_COLOR=1 uv export --format requirements.txt --no-hashes --no-dev > requirements.txt` — without the `NO_COLOR` flag, colour escape codes get written into the file ahead of every comment marker, silently breaking every comment line and failing a plain `pip install` on line one, invisibly in any editor that renders colour.

Frontend

Standard Vite dev server; there is no committed `.env` in `frontend/`, and the dev server proxies `/api` requests to the backend on port 8000. Type checking must be run as `npm run tsc -b` — plain `npm run tsc --noEmit` silently compiles an empty project and exits successfully regardless of real type errors, because the root `tsconfig.json` is a project-reference solution file with no files of its own.

Machine learning

Its own `uv` virtual environment and `requirements.txt`, entirely separate from the backend's — it is never imported by the running API. See its own `README.md` for pipeline-specific commands.

Configuration & Credentials

All external-service credentials live in a single `Settings` object in the backend's config module, each with an empty-string default — real values are never committed and belong only in an untracked `backend/.env` file. This covers Copernicus Marine, Global Fishing Watch, and the Gmail SMTP credentials the feedback form needs to actually send mail. The machine learning pipelines read credentials from that same `backend/.env` file rather than maintaining a second secrets location.

Known Limitations & Roadmap

Area	Current state	What's intentionally deferred
Eddy detection	Per-frame detection, stateless	Frame-to-frame tracking (age, trajectory) — a real assignment problem, not yet attempted, to avoid shipping a flickering "track" that looks like an observation
Drift simulation	Single snapshot-based path	A time-indexed forecast with an uncertainty envelope, rather than one deterministic line
Upwelling corroboration	Measured weak agreement, reported honestly	A climatology fitted on the live reanalysis product instead of the satellite record it's currently built on — tested and found worse in its current form, not yet solved
Ammonium / tidal height	Marked unavailable	No known global source currently exists for either
Core/observations DB schema	Migrated, unused	Waiting on a feature that genuinely needs persisted records rather than configuration

PART V

Appendix

Reference tables, a glossary of terms used throughout this guide, and answers to the questions that come up most often.

Data Source Directory

Source	Provides	Access
Copernicus Marine	SST, currents, waves, salinity, biogeochemistry, wind	Credentialed, server-side only
Global Fishing Watch	Vessel activity (AIS)	API token, server-side only
GEBCO / Ifremer ERDDAP	Seafloor bathymetry	Open, no key
Open-Meteo	Weather and marine point forecasts	Open, no key
NOAA Coral Reef Watch	Bleaching risk, heat-stress climatology	Open ERDDAP
OBIS	Species occurrence, eDNA detections	Open REST API
GBIF	Species occurrence (research-grade candidate, not yet merged into production ML)	Open REST API

Glossary

Term	Meaning
Eddy	A rotating current structure; the map's detector flags one where rotation dominates strain in the current field, relative to that field's own variance
Cyclonic / anticyclonic	Rotation relative to the planet's spin; the same rotation direction is classified oppositely in the northern and southern hemispheres
Upwelling	Wind-driven replacement of surface water with colder, nutrient-rich deep water along a coast
eDNA	Environmental DNA — species identified from genetic material in a water sample rather than a physical sighting
Skill score	A forecast model's accuracy measured against a naive persistence baseline, not in isolation
Climatology	The expected seasonal value for a given day of year, fitted from historical data, used as a second forecasting baseline
Persistence (baseline)	The naive forecast "tomorrow will equal today" — the floor every real model must beat
Delta target	Training a model to predict the change in a variable rather than its absolute future value
Grounded (chat)	An assistant answer whose every number was verified against the platform's own tool results after the answer finished generating
Grid / forecast grid	A precomputed global raster of the forecasting engine's prediction, rendered as a map layer
Bakun index	A wind-derived index of upwelling favourability
Marine Data Fusion Layer	The shared regridding and feature-store layer both offline ML pipelines are built on

Frequently Asked Questions

Is the data on the map really live?

Yes. Every layer is fetched from its real upstream provider on a schedule appropriate to how fast that source actually updates, cached server-side, and served from that cache — never generated or simulated for the demo.

Why does a layer sometimes say "unavailable" instead of just showing something?

Because showing a plausible-looking placeholder value would be a false ocean reading. MarisAI's core rule is to always say when data is missing rather than substitute something that looks like data.

Can I trust the AI assistant's numbers?

Trust the ones marked "grounded" — they've been checked against the raw tool results that produced the answer. Treat streaming, in-progress text as provisional until that check completes, and treat any explicitly flagged figure with real scepticism; it means the check found no source for it.

Why can't I download tidal height or ammonium data?

No global-coverage source for either currently exists among this platform's providers — it's a real, stated gap, not an oversight in the export tool.

Does the forecasting model know what it's talking about, or is it guessing?

Every shipped forecast horizon has cleared a real bar: it beats a naive "no change" baseline, and it does so consistently across most cross-validation folds, not just on average. Horizons that don't clear that bar are not shipped, and the ones that failed are kept on file as a record rather than deleted.

What's the difference between the Dashboard and Analytics pages?

The Dashboard is a global, multi-variable overview built for a quick scan. The Analytics page is the original point-focused deep-dive tool — it answers a narrower, single-location question the dashboard isn't built to.

MarisAI — The Complete Guide
Prepared August 2026